

GPU-Accelerated Isoalgorítmico GA+2-opt para TSP

Lucas Galdino

2025-11-22

- Capítulo 1 – Introdução
 - 1.1 Contexto e motivação
 - 1.2 Problema de pesquisa
 - 1.3 Objetivo geral
 - 1.4 Objetivos específicos
 - 1.5 Justificativa
- Capítulo 2 - Revisão bibliográfica
 - 2. Fundamentação teórica e revisão bibliográfica
 - * 2.1 Otimização combinatória e o Problema do Caixeiro Viajante
 - * 2.2 Heurísticas para o TSP
 - * 2.3 Algoritmos genéticos e heurísticas híbridas
 - * 2.4 Computação em GPU e paralelização de meta-heurísticas
 - * 2.5 Comparação estatística de algoritmos de otimização

Capítulo 1 – Introdução

1.1 Contexto e motivação

O Problema do Caixeiro Viajante (Traveling Salesman Problem – TSP) é um dos problemas mais estudados em otimização combinatória (1). Tem esse nome devida a sua história clássica: um vendedor precisa visitar um conjunto de cidades, passando por cada uma exatamente uma vez, e deseja minimizar a distância total percorrida.

O objetivo é encontrar um ciclo hamiltoniano (ciclo onde deve-se passar por todos os vértices uma vez e retornar ao vértice original). Apesar de sua formulação simples, o TSP é NP-difícil (que é um problema sem solução em tempo polinomial) e, na sua formulação de decisão: “existe um tour com custo menor ou igual a B ?”, é NP-completo: não se conhece algoritmo em tempo polinomial que o resolva em geral, embora seja fácil verificar o custo de uma solução candidata. Cook (1) argumenta que essa combinação de simplicidade, dificuldade teórica e rica estrutura geométrica faz do TSP um estudo de caso central tanto para a teoria da complexidade quanto para desenvolvimento de métodos exatos e heurísticos.

Na prática, variantes do TSP aparecem em domínios como roteirização de veículos, planejamento de inspeções, manufatura e testes de circuitos, genômica e astronomia, entre outros (1). Exemplos clássicos incluem o posicionamento e ordenamento de furos em placas de circuito impresso, logística e problemas de mapeamento genético. Mesmo quando modelos reais são mais complexos (com janelas de tempo, múltiplos veículos ou restrições de capacidade), é comum validar ideias de projeto e análise de algoritmos primeiro em instâncias clássicas do TSP, justamente pela ampla disponibilidade de referências, testes padronizados e de soluções ótimas conhecidas.

Paralelamente, a evolução do hardware trouxe processadores gráficos (GPUs) como plataforma acessível para computação de alto desempenho (HPC) (2) e com a popularização e acessibilidade, pesquisas nestas áreas cresceram rapidamente. As IAs generativas são totalmente dependentes desse tipo de hardware, por exemplo.

GPUs oferecem milhares de núcleos relativamente simples, organizados em um modelo de execução massivamente paralelo, mais próximo do paradigma SIMD/SIMT descrito por Flynn e extensões modernas (3,4). Em troca de um controle mais restrito nos fluxos de processo e memória, essas arquiteturas entregam uma grande largura de banda de memória (comunicação entre CPUs e GPUs) e uma grande taxa de operações aritméticas por segundo (FLOPs), desde que o problema ofereça operações semelhantes que

possam ser executadas em paralelo, sejam elas utilizando paralelismos funcionais (mais complexos) e paralelismos de dados (mais simples). Isso torna GPUs particularmente atrativas para tarefas como o cálculo de matrizes (ganho exponencial, como demonstrado nesse trabalho), a avaliação de grandes populações de soluções e a aplicação de movimentos de vizinhança independentes em heurísticas de melhoria.

Do ponto de vista das meta-heurísticas, Crainic e Toulouse propõem uma taxonomia de paralelização que distingue, em linhas gerais, paralelismo em dados, paralelismo funcional e esquemas com múltiplas trajetórias cooperativas (5,6). De forma simplificada, pode-se falar em três tipos:

1. paralelismo *dados* (por exemplo, várias soluções de uma população sendo avaliadas em paralelo);
2. paralelismo em *tarefas* dentro de uma mesma trajetória de busca (por exemplo, vizinhanças sendo exploradas em paralelo para uma solução corrente);
3. paralelismo que mantém várias trajetórias completas de busca (estratégias de início simultâneo, modelos em ilhas, busca cooperativa).

O projeto desta monografia explora principalmente algoritmos do tipo 1, ao explorar paralelismo em dados em vizinhanças de 2-opt e na avaliação de populações em algoritmos genéticos, em linha com estudos recentes sobre heurísticas para TSP em GPU (7,8,9).

Ainda assim, nem todo algoritmo se beneficia automaticamente de uma migração para GPU. Inicializações de kernels, movimentação de dados entre CPU e GPU podem anular ganhos de paralelismo, especialmente em problemas de porte pequeno ou em implementações que realizam pouco trabalho por inicializações de kernel (10,11). Limitações de memória (VRAM) devem ser cuidadosamente calculadas para que a memória total ocupada não passe do limite do hardware. Nesses cenários, uma comparação direta entre versões em CPU e GPU exige cuidado extra: deve-se isolar o impacto da plataforma de execução sem confundir o efeito com mudanças na lógica do algoritmo. Ao longo deste trabalho, esses desafios são documentados explicitamente em estudos de caso, onde kernels são chamados de forma não cautelosa e versões otimizadas (Capítulos 3 e 4), bem como em notas técnicas específicas sobre *overhead* de lançamentos de kernel e padrões de redução em GPU.

Este trabalho insere-se nesse contexto, investigando como acelerar, utilizando GPU, um Algoritmo Genético (Genetic Algorithm – GA) híbrido com 2-opt para o TSP, de forma controlada e estatisticamente rigorosa, >[!caution] > - que métodos? ainda não analisamos os dados, nem sabemos se realmente iremos utilizar todos. > - deixar esta nota aqui até resolução. usando instâncias clássicas da biblioteca TSPLIB (12). A combinação entre GA e busca local é um exemplo de algoritmo memético amplamente estudado na literatura (13,14), em que operadores evolutivos globais são complementados por heurísticas de melhoria como 2-opt (15) ou Lin–Kernighan (16). Neste Trabalho de Conclusão de Curso, usarei a nomenclatura em inglês para alguns termos técnicos como “2-opt”, “TSPLIB”, “GPU”, “CPU” e “GA”.

1.2 Problema de pesquisa

Muitos estudos em computação de alto desempenho comparam algoritmos em CPU e GPU utilizando implementações que diferem não apenas na plataforma de execução, mas também em detalhes relevantes da lógica do algoritmo (por exemplo, operadores distintos, parâmetros diferentes ou vizinhanças não equivalentes) (7,10,17). Isso torna difícil atribuir ganhos de desempenho exclusivamente ao uso da GPU, uma vez que alterações no desenho algorítmico ou na parametrização podem, por si só, explicar diferenças observadas. Parâmetros escolhidos serão discutidos no capítulo 3 >[!caution] fazer o link para pular para o capítulo 3, de forma que seja renderizada em pandoc como um link em pdf no latex

Neste trabalho, busquei implementar os algoritmos sob as mesmas condições. A estrutura dos algoritmos busca ser idêntica e otimizada tanto para CPU, quanto para GPU, de forma justa, para todos os Algoritmos Genéticos, assim como para o algoritmo de busca local 2-opt, variando apenas a forma como cada “módulo” estrutural do algoritmo é inserido durante o processo de execução e como ele será executado: Através de paralelismo ou não. A pergunta central pode ser formulada da seguinte forma:

Como comparar, de forma fiel, o impacto de diferentes estratégias de paralelização utilizando GPU no desempenho de um Algoritmo Memético (GA+2-opt) >[!caution] Introduzir o conceito de algoritmo memético mais acima e referenciar para o Problema do Caixeiro Viajante?

Responder a essa pergunta exige, ao mesmo tempo, um desenho experimental cuidadoso de hiperparâmetros e validade estatística (número significativo de instâncias, número adequado de repetições, métricas

de qualidade e tempo). Os parâmetros básicos dos GAs (tamanho de população n_{pop} , taxa de mutação m >[!caution] adicionar os símbolos matemáticos próprios ao katex, usar o símbolo conforme literatura para taxa de mutação, torneio k , iterações de 2-opt) n_{rep} são escolhidos com base em recomendações da literatura de algoritmos evolutivos (18,14) e em análises específicas documentadas nos relatórios técnicos do projeto (Capítulo 3). >[!caution] fazer o link para pula para o capítulo 3, de forma que seja renderizada em pandoc como um link em pdf no latex

1.3 Objetivo geral

O objetivo geral deste trabalho é:

Investigar e quantificar, o impacto de diferentes estratégias de paralelização em GPU, seguindo a taxonomia de (19) -> tipos 1,2,3 sobre o tempo de execução e a qualidade das soluções de um Algoritmo Genético híbrido com 2-opt aplicado ao Problema do Caixeiro Viajante.

[!caution] mais detalhado. Objetivo muito fora do padrão, traduzido ao pé da letra. Objetivo não

[!ULTRACAUTION] a citação de autores deve ser feita corretamente, com o sobrenome e o ano, conforme as normas ABNT. Deve assegurar que as fontes realmente dizem isso. Também, senti falta dos documentos principais e mais densos. #read-pdf crainic2003parallel, #websearch and #fetch article{crainic2010parallel designing_parallel_metaheuristics. Não há paenas paralelismo funcional e

1.4 Objetivos específicos

Para viabilizar o objetivo geral, definem-se os seguintes objetivos específicos:

1. **Implementar quatro variações isoalgorítmicas** de um Algoritmo Genético híbrido com 2-opt para o TSP, que diferem apenas na forma e no local de execução (CPU ou GPU):
 - uma versão executada puramente no CPU (**GA-CPU**), em que todas as operações (seleção, cruzamento, mutação, 2-opt e cálculo de custo) são executadas com NumPy, servindo como referência sequencial; linkar com o capítulo 3 >[!caution] fazer o link para pula para o capítulo 3, de forma que seja renderizada em pandoc como um link em pdf no latex; Tem certeza que são are ferência sequencial? leia [full_benchmark results](./code/benchmarks/results/full_benchmark.log);
 - uma versão híbrida com 2-opt executada na GPU e avaliação em CPU (**GA-Híbrido-TransferênciaCompleta**), que inicia kernels **individualmente** para cada rota e transfere as soluções (ou tours) completos entre CPU e GPU a cada iteração;
 - uma versão híbrida otimizada (**GA-Híbrido-Batch**), em que 2-opt e o cálculo de custos são executados em GPU em lote (*batch*), reduzindo o número de lançamentos de kernel e o volume de dados transferidos;
 - uma versão totalmente em GPU (**GA-FullGPU**), na qual população, operadores genéticos, busca local e avaliação permanecem residentes na GPU durante toda a evolução.

[!caution]

 - id: chap3
 - description: Fazer o link para o capítulo 3, onde a arquitetura do framework. Não gostei dos nomes. Deve ter diagrama
 - mantenha essa nota aqui até resolução.
2. **Selecionar um conjunto de instâncias da TSPLIB** com diferentes faixas de tamanho (pequenas $n_{coords} \leq 100$, médias $100 < n_{coords} \leq 500$ e grandes $n_{coords} > 500$), de forma a demonstrar os imensos ganhos de performance que podem ser atingidos e tendo diversidade suficiente para análise de ganhos em escala, tendo apenas um leve ajuste na prática de agrupamento utilizada em estudos prévios com TSPLIB (12).
3. **Definir um protocolo experimental reprodutível**, incluindo número de repetições por instância e algoritmo, critérios de parada, parâmetros do GA e limites práticos impostos pela capacidade

de memória da GPU. O desenho desse protocolo segue recomendações da literatura de comparação de algoritmos e de testes estatísticos (20) e é detalhado no Capítulo 3 [Link with chapter 3](#).

4. **Aplicar um conjunto de testes estatísticos apropriados** para comparação de algoritmos, incluindo testes de normalidade (Shapiro–Wilk), testes pareados paramétricos (teste t pareado) e não paramétricos (Wilcoxon), análise de rankings em múltiplas instâncias (teste de Friedman com pós-teste de Nemenyi) e medidas de tamanho de efeito (Cohen d), conforme recomendações em (20) e literatura correlata de teste de hipóteses. >[!caution] > - essa nota fica aqui até resolução final sobre o algoritmo utilizado. Algoritmo base é, depois de 100 iterações, o **GA_{naive} + 2-opt**. > - Adicionar ao capítulo 3 a seção de metodologia estatística utilizada.
5. **Analisar os resultados obtidos**, discutindo as possíveis condições (tamanho da instância n_{coords} , relação entre custo de comunicação e custo de computação padrões de acesso à memória) >[!caution] >add proper latex and prper reference for nvidiacuda D2H, H2D. Appendix could have code/pseudocode.

cada variação isoalgorítmica é vantajosa, os limites práticos do uso de GPU neste contexto e as implicações para o projeto de algoritmos de roteamento em cenários reais, à luz das diretrizes para paralelização de meta-heurísticas (5,21) e de estudos de caso em roteamento em GPU (7,8,9).

1.5 Justificativa

Do ponto de vista científico, o TSP continua sendo um problema de referência para avaliar novas ideias em heurísticas e meta-heurísticas (1). A vasta disponibilidade de estudos, instâncias utilizadas em larga escala e muitas soluções ótimas conhecidas, em particular as fornecidas pela TSPLIB (12), na qual o projeto se baseia, permite medir o desempenho de diferentes algoritmos tanto em termos de qualidade quanto em tempo de execução. >[!caution] > >- adicionar variáveis katex em inglês com subíndices. Pode introduzir depois, mas algumas já podem ir sendo pré-definidas. Usar $n_{\{coords\}}$ para número de coordenadas, $n_{\{pop\}}$ para tamanho de população. variáveis para transferência de dados, tempo de execução, etc. >- id: chap3 >- description: adicionar cálculos e explicação para controle de memória de cada algoritmo no capítulo 3

No campo da computação de alto desempenho, diferentes trabalhos relatam ganhos relevantes ao portar heurísticas de melhoria e algoritmos evolutivos para GPU, inclusive em variantes do TSP (7,8,9). Esses resultados indicam que GPUs são um hardware promissor para esse tipo de algoritmo, mas também evidenciam um problema recorrente: em muitas comparações, as versões em CPU e GPU diferem não apenas na plataforma de execução, mas também em detalhes de implementação e parametrização, o que dificulta isolar o efeito específico da paralelização.

Além disso, para que conclusões sobre desempenho sejam confiáveis, não basta observar poucos experimentos isolados. É necessário adotar um desenho experimental com múltiplas repetições, tanto para instâncias quanto para, medidas de dispersão e testes de hipótese adequados, como discutido na literatura de comparação de algoritmos (20). Neste trabalho, esses cuidados são incorporados ao experimento, de modo que as diferenças observadas entre variantes em CPU e GPU possam ser atribuídas, com maior segurança, às estratégias de paralelização avaliadas.

Este trabalho justifica-se, portanto, por combinar três elementos que, em conjunto, fortalecem a contribuição científica:

1. **Implementação de heurísticas clássicas em GPU** (particularmente 2-opt e operadores do AG), explorando o paralelismo de forma explícita.
2. **Comparação entre variantes funcionalmente equivalentes do algoritmo em CPU e GPU**, mantendo fixos operadores, parâmetros e critérios de parada, de modo a isolar o efeito da plataforma de execução.
3. **Aplicação de metodologia estatística rigorosa**, com múltiplas repetições por instância, testes de hipótese adequados e medidas de tamanho de efeito.

Os capítulos seguintes discutem, à luz desses três eixos, em que condições cada variante é vantajosa, quais são os limites práticos do uso de GPU neste contexto e como os resultados dialogam com estudos prévios de heurísticas em GPU.

[!note] ainda verei se usarei ou não essa nota

1.6 Organização do traba

Este texto está organizado da seguinte forma:

- **Capítulo 2 – Fundamentação Teórica / Revisão Bibliográfica:** apresenta os conceitos básicos de otimização combinatória e do TSP, revisa heurísticas de construção e melhoria (com ênfase em 2-opt), discute algoritmos genéticos e meta-heurísticas híbridas, introduz princípios de computação em GPU e paralelização de meta-heurísticas, e resume abordagens estatísticas para comparação de algoritmos.
- **Capítulo 3 – Materiais e Métodos:** descreve o ambiente computacional, a arquitetura do framework desenvolvido (incluindo abstrações de backend e estratégias isoalgorítmicas), os detalhes do Algoritmo Genético híbrido com 2-opt nas quatro variantes consideradas, o conjunto de instâncias TSPLIB utilizado, os parâmetros experimentais e o protocolo estatístico adotado.
- **Capítulo 4 – Resultados:** apresenta os resultados numéricos dos experimentos, incluindo estatísticas por instância, análises agregadas por faixa de tamanho, medidas de speedup e qualidade das soluções, além da interpretação dos testes estatísticos aplicados.
- **Capítulo 5 – Discussão:** interpreta criticamente os resultados à luz da literatura, discute as implicações dos achados para o projeto de algoritmos de roteamento em GPU e analisa limitações do estudo.
- **Capítulo 6 – Conclusões e Trabalhos Futuros:** sintetiza as principais contribuições do trabalho, responde explicitamente aos objetivos propostos e indica possíveis extensões, como a aplicação do framework a problemas de roteirização mais complexos.
- Elementos pós-textuais, como referências, glossário, apêndices técnicos e anexos com tabelas completas de resultados, são apresentados ao final do documento, conforme normas da instituição.

Capítulo 2 - Revisão bibliográfica

2. Fundamentação teórica e revisão bibliográfica

Este capítulo apresenta conceitos teóricos e a bibliografia utilizada para contextualizar o problema estudado e as escolhas metodológicas adotadas. São discutidos o Problema do Caixeiro Viajante (TSP) no contexto da otimização combinatória(22), sua importância geral e exemplos de utilização. Também serão discutidas generalizações do problema(22), heurísticas e metaheurísticas clássicas para o TSP(23), aprofundando-se então no tópico de algoritmos genéticos e algoritmos meméticos(24), princípios de computação em GPU(22) e taxonomias de paralelização de meta-heurísticas(24), além de noções básicas de comparação estatística de algoritmos de otimização(24). >[!caution] >por favor, adicionar as referências aqui, para cada situação

2.1 Otimização combinatória e o Problema do Caixeiro Viajante O TSP pode ser formulado, na versão simétrica¹, como um problema de encontrar um ciclo hamiltoniano de custo mínimo — podendo este ser a distância percorrida, combustível gasto, ou mesmo uma combinação de fatores. Para fins deste trabalho, o custo será também chamado de distância — em um grafo completo não direcionado $G = (V, E)$, no qual cada vértice em V representa uma cidade e cada aresta $(i, j) \in E$ possui um custo $c_{ij} \geq 0$ entre as cidades i e j (1). O objetivo é determinar uma combinação das cidades que minimize a soma total dos custos, retornando à cidade de origem. Geralmente em aplicações práticas assume-se que a matriz de custos é métrica e euclidiana, isto é, os custos derivam de distâncias euclidianas entre coordenadas no plano.

Do ponto de vista da complexidade, o TSP é um problema NP-difícil em sua forma de otimização e NP-completo em sua forma de decisão (1). Isso significa que, em geral, não se conhece um algoritmo em tempo polinomial que resolva instâncias arbitrárias em grande escala. Mesmo que seja relativamente fácil verificar o custo de uma ou várias soluções candidatas, não podemos afirmar que esta é solução a exata em tempo polinomial. Algoritmos modernos como Held-Karp (25) com uma complexidade $O(n^2 2^n)$ e o algoritmo Concorde (1) com uma complexidade indefinida, conseguem resolver instâncias com milhares

¹Na versão assimétrica do TSP, os custos de viagem entre pares de cidades podem diferir dependendo da direção (ou seja, $c_{ij} \neq c_{ji}$). Em TSPs simétricos, o custo do cálculo de distâncias pode ser representadas por uma matriz triangular, diminuindo os cálculos de n^2 para $(n)(n+1)/2$. >[!caution] >Eu não tenho certeza da fórmula, por favor, checar.

de cidades ², mas seu tempo de execução cresce exponencialmente (ainda melhor que a força bruta $(n-1)!$) com o tamanho do problema, tornando-os impraticáveis para instâncias muito grandes.

Ao longo das últimas décadas, o TSP consolidou-se também como um padrão de avaliação empírica de algoritmos, graças à disponibilidade de coleções de instâncias padronizadas, como a TSPLIB (12). Essas coleções incluem instâncias com diferentes tamanhos, estruturas e origens (geográficas, sintéticas, industriais). A maioria delas com soluções ótimas conhecidas e obtidas por métodos exatos. No presente trabalho, são utilizadas algumas dessas instâncias como base para avaliar e comparar heurísticas e variantes paralelas de um Algoritmo Memético (algoritmos evolucionários a técnicas de busca locais) (24).

2.2 Heurísticas para o TSP Devido à dificuldade de resolver instâncias grandes do TSP exatamente, uma vasta literatura de heurísticas tem sido desenvolvida para produzir boas soluções em tempos de computação aceitáveis (26). De forma geral, heurísticas podem ser agrupadas em dois grandes tipos: heurísticas de construção e heurísticas de melhoria.

Heurísticas de construção produzem uma solução viável “do zero”, frequentemente seguindo regras simples, também algoritmos gulosos, como escolher iterativamente o vizinho mais próximo ou inserir cidades em posições que causem o menor aumento de custo. Exemplos incluem a heurística do vizinho mais próximo (nearest neighbor — NN), heurísticas de inserção (insertion heuristics) e variantes baseadas em árvores de extensão mínimas (Minimum Spanning Trees — MSTs), como o Algoritmo de Christofides. Embora rápidas e fáceis de implementar, essas estratégias tendem a gerar soluções de qualidade moderada, servindo sobretudo como ponto de partida para métodos mais sofisticados. Simchi et al. (27) demonstra que algumas dessas soluções garantem uma solução H^* não maior que $1.5 \times$ o custo ótimo B^* , em casos específicos. >[!caution] Citações necessárias > citar ainda (27) e os cálculos de upper bounds e lower bounds relacionados -> capítulos 4 e 5 do livro. #read-pdf simchi-levi

Heurísticas de melhoria partem de uma solução inicial e aplicam sucessivos movimentos locais que procurem reduzir seu custo. Entre essas, as chamadas k -opt são particularmente influentes: um movimento 2-opt consiste em remover duas arestas de um tour e reconectar os segmentos resultantes de modo a eliminar cruzamentos e reduzir o comprimento total (15); >[!caution] >2-opt movements really only affect crossings? As long as I know, it does not affect only crosses. Please, better explain this.

movimentos 3-opt e extensões mais complexas, como o algoritmo de Lin–Kernighan, generalizam essa ideia (16). Estas e outras heurísticas de melhoria desempenham papel central em muitos algoritmos modernos para TSP, tanto como procedimentos isolados quanto como componentes de meta-heurísticas, sendo utilizados como procedimentos de busca local.

Neste trabalho, a heurística 2-opt é utilizada como busca local básica acoplada ao Algoritmo Genético, em linha com diversos estudos que combinam heurísticas de construção simples com procedimentos de melhoria mais intensivos para obter soluções de alta qualidade em tempo razoável (28).

2.3 Algoritmos genéticos e heurísticas híbridas Algoritmos Genéticos (Genetic Algorithms – GAs) são meta-heurísticas inspiradas em princípios de evolução biológica, nas quais uma população de soluções candidatas é iterativamente modificada por operadores análogos à seleção natural, recombinação e mutação (14,18). Na sua forma mais simples, um AG mantém uma população de indivíduos representando soluções para o problema; em cada geração, indivíduos são selecionados com base em uma função de aptidão (fitness), recombinados por operadores de cruzamento e perturbados por operadores de mutação. A nova população resultante substitui total ou parcialmente a anterior, e o processo se repete até que um critério de parada seja satisfeito.

No contexto do TSP, é comum representar cada solução como uma permutação das cidades, utilizar operadores de cruzamento especializados para rotas — como Order Crossover (OX), Partially Matched Crossover (PMX), entre outros — e empregar mutações que preservem a viabilidade da permutação, como trocas de posição entre duas cidades (13). Estudos clássicos mostram que GAs podem produzir soluções competitivas para o TSP quando combinados com operadores e parâmetros adequados (14).

²O algoritmo Held-Karp “troca” a complexidade temporal da força bruta $O(n!)$ por uma complexidade espacial $O(n2^n)$, tornando sua viabilidade limitada a ~ 40 problemas. Já o algoritmo Concorde, baseado em técnicas de ramificação e corte, é capaz de resolver instâncias com até 85.900 cidades, mas seu desempenho depende fortemente da estrutura específica da instância (1).

Uma extensão importante dessa ideia é o conceito de algoritmos meméticos, nos quais operadores evolutivos (seleção, cruzamento, mutação) são combinados com heurísticas de busca local aplicadas a indivíduos da população (13). Em problemas de roteamento, isso resulta em esquemas GA+LS, nos quais um GA guia a exploração global do espaço de soluções e um procedimento de melhoria, como 2-opt ou Lin-Kernighan, refina soluções promissoras. Trabalhos como o de Fujimoto e Tsutsui (9) tiveram uma grande influência na decisão do tema e exploram justamente essa combinação para o TSP em ambientes de computação paralela, motivando a adoção de uma abordagem semelhante neste estudo.

2.4 Computação em GPU e paralelização de meta-heurísticas Processadores gráficos (GPUs) evoluíram, nas últimas décadas, de dispositivos voltados principalmente para renderização gráfica para plataformas de computação de uso geral (GPGPU), amplamente utilizadas em aplicações científicas e de inteligência artificial (2). O modelo de programação CUDA, por exemplo, organiza o trabalho em grades (*grids*) de blocos de threads, seguindo um paradigma de execução massivamente paralelo próximo ao SIMT (Single Instruction, Multiple Threads), relacionado às classificações de arquiteturas de Flynn (3,4). Nessa configuração, milhares de threads executam o mesmo kernel sobre dados distintos, o que é adequado a tarefas com alto grau de paralelismo em dados. >[!caution] >revisar amanhã

Crainic e Toulouse propuseram uma taxonomia para paralelização de meta-heurísticas que as distingue em três grandes tipos: abordagens que exploram paralelismo em dados (neste trabalho, usadas intercaladamente como *P-Data*), paralelismo em tarefas (neste trabalho, usadas intercaladamente como *P-Tasks*) e esquemas com trajetórias de busca cooperativas (5,6). No **tipo 1**, avaliações de soluções, cálculos de custo e procedimentos de busca local, principalmente quando cálculos de matrizes estão envolvidas, são distribuídos entre vários processadores (paralelismo em CPU, por exemplo usando os vários núcleos do processador) ou threads (GPUs, modelo SIMT). No **tipo 2**, diferentes partes ou fases de um mesmo algoritmo são executadas em paralelo; no **tipo 3**, múltiplas execuções (trajetórias) de uma meta-heurística interagem por meio de mecanismos de cooperação, como modelos em ilhas ou memórias compartilhadas. >[!caution] >preciso revisar essa part amanha, sobre os tipos 1,2,3. >Parece ir contra o que eu tinha entendido antes.

Aplicações de GPU a meta-heurísticas para o TSP exploram principalmente paralelismo em dados, seja na avaliação em massa de rotas em algoritmos genéticos (através de cálculos de redução), seja aplicando paralelismo a movimentos de vizinhança em heurísticas de melhoria (7,8,9). Outros trabalhos investigam paralelização de Simulated Annealing, Colônias de Formigas (ACO) e outras meta-heurísticas em GPU, geralmente aproveitando o grande número de threads para explorar múltiplas soluções ou vizinhanças em cada passo da busca (28). Esses estudos motivam o uso de GPUs como plataforma para acelerar algoritmos meméticos,

mas também evidenciam desafios relacionados a movimentação de dados entre CPU e GPU, escolha de gran

[!caution] adicionar mais informações sobre esses desafios, linkar com o próximo capítulo.

2.5 Comparação estatística de algoritmos de otimização Meta-heurísticas estocásticas, como GAs e algoritmos meméticos, produzem resultados que variam de execução para execução devido ao uso de aleatoriedade em diversos pontos (inicialização, seleção, mutação, entre outros). Por esse motivo, a comparação de algoritmos de otimização não pode se basear em uma única execução por instância, tampouco apenas em médias simples sem qualquer análise de variabilidade. A literatura de comparação de algoritmos enfatiza a importância de utilizar múltiplas instâncias de teste, múltiplas repetições por combinação algoritmo-instância e métricas que considerem tanto qualidade da solução quanto tempo de execução (20).

Demšar (20) discute procedimentos estatísticos apropriados para comparar algoritmos em vários problemas, recomendando o uso de testes de hipótese pareados (como o teste *t* pareado ou o teste de Wilcoxon) quando se deseja comparar dois algoritmos em uma coleção de instâncias, e testes baseados em rankings, como o teste de Friedman seguido de pós-testes de Nemenyi, quando mais de dois algoritmos são avaliados simultaneamente. Medidas de tamanho de efeito, como o *d* de Cohen, complementam a análise ao quantificar a magnitude prática das diferenças observadas.

Neste trabalho, esses princípios gerais orientam o desenho experimental e a análise dos resultados apresentados nos capítulos seguintes, garantindo que as comparações entre variantes em CPU e GPU sejam realizadas de forma estatisticamente fundamentada.

Referências

- [1] COOK, W. J. **In Pursuit of the Traveling Salesman: Mathematics at the Limits of Computation.** [s.l.] Princeton University Press, 2012.
- [2] NVIDIA CORPORATION. **CUDA C++ Best Practices Guide.** [s.l.: s.n.].
- [3] DRAFT. **DRAFT: flynn1972taxonomy.**, 2025.
- [4] DRAFT. **DRAFT: almasi2002high.**, 2025.
- [5] CRAINIC, T. G.; TOULOUSE, M. Parallel Strategies for Meta-Heuristics. Em: GLOVER, F.; KOCHENBERGER, G. A. (Eds.). **Handbook of Metaheuristics.** Boston, MA: Springer, 2003. p. 475–513.
- [6] CRAINIC, T. G.; TOULOUSE, M. Parallel meta-heuristics. Em: **Handbook of metaheuristics.** [s.l.] Springer, 2010. p. 497–541.
- [7] SCHULZ, C. et al. GPU computing in discrete optimization. Part II: Survey focused on routing problems. **EURO journal on transportation and logistics**, v. 2, n. 1-2, p. 159–186, 2013.
- [8] ROCKI, K.; SUDA, R. **High Performance GPU Accelerated Local Optimization in TSP.** Proceedings of the 2013 IEEE 27th International Symposium on Parallel and Distributed Processing Workshops and PhD Forum. **Anais...: IPDPSW '13.** Washington, DC, USA: IEEE Computer Society, 2013. Disponível em: <<http://dx.doi.org/10.1109/IPDPSW.2013.227>>
- [9] FUJIMOTO, N.; TSUTSUI, S. **A highly-parallel tsp solver for a gpu computing platform.** Numerical Methods and Applications: 7th International Conference, NMA 2010, Borovets, Bulgaria, August 20-24, 2010. Revised Papers 7. **Anais...Springer**, 2011.
- [10] VAN LUONG, T.; MELAB, N.; TALBI, E.-G. GPU Computing for Parallel Local Search Metaheuristic Algorithms. **IEEE Transactions on Computers**, v. 62, n. 1, p. 173–185, 2013.
- [11] LUONG, T. V.; MELAB, N.; TALBI, E.-G. **GPU-based island model for evolutionary algorithms.** Proceedings of the 15th annual conference on Genetic and evolutionary computation. **Anais...2013.**
- [12] REINELT, G. TSPLIB—A Traveling Salesman Problem Library. **ORSA Journal on Computing**, v. 3, n. 4, p. 376–384, 1991.
- [13] LARRAÑAGA, P. et al. Genetic algorithms for the travelling salesman problem: A review of representations and operators. **Artificial Intelligence Review**, v. 13, n. 2, p. 129–170, 1999.
- [14] GOLDBERG, D. E. **Genetic Algorithms in Search, Optimization, and Machine Learning.** [s.l.] Addison-Wesley, 1989.
- [15] CROES, G. A. A method for solving traveling-salesman problems. **Operations Research**, v. 6, n. 6, p. 791–812, 1958.
- [16] LIN, S.; KERNIGHAN, B. W. An effective heuristic algorithm for the traveling-salesman problem. **Operations Research**, v. 21, n. 2, p. 498–516, 1973.
- [17] BENAINI, A.; BERRAJAA, A. **Genetic algorithm for large dynamic vehicle routing problem on GPU.** 2018 4th International Conference on Logistics Operations Management (GOL). **Anais...IEEE**, 2018.

- [18] EIBEN, A. E.; SMITH, J. E. **Introduction to Evolutionary Computing**. 2nd. ed. [s.l.] Springer, 2015.
- [19] DRAFT. **DRAFT: Crainic2003.**, 2025.
- [20] DRAFT. **DRAFT: demsar2006statistical.**, 2025.
- [21] ALBA, E. **Parallel metaheuristics: a new class of algorithms**. [s.l.] John Wiley & Sons, 2005.
- [22] DRAFT. **DRAFT: properref.**, 2025.
- [23] DRAFT. **DRAFT: roperrefs.**, 2025.
- [24] DRAFT. **DRAFT: properrefs.**, 2025.
- [25] DRAFT. **DRAFT: held1962dynamic.**, 2025.
- [26] DRAFT. **DRAFT: otherrefs.**, 2025.
- [27] SIMCHI-LEVI, D. et al. The logic of logistics. **Theory, algorithms, and applications for logistics and supply chain management**, 2005.
- [28] DRAFT. **DRAFT: addproperrefs.**, 2025.